

8.6 - Modularizando o Monólito

Do monólito acoplado ao monólito modular

Modularizar antes de extrair

O passo que quase todo mundo pula

- **Nem todo monólito precisa virar microserviços**
 - Um monólito modular saudável já resolve a maior parte
- **Extrair serviços de um monólito acoplado é receita de caos**
 - Sem fronteiras claras, você só distribui o emaranhado
- **Primeiro organize por dentro, depois separe se precisar**
 - Modularização é pré-requisito de qualquer extração segura
- **O alvo é o monólito modular, não o microserviço**
 - Microserviço prematuro adiciona rede a um problema de design

O que sobrou do monólito vira modular

- **Billing e configurações de cliente ainda no monólito**
 - Modularizados antes de qualquer extração

LUNALOG

O que sobrou do monólito da LunaLog, billing e configurações de cliente, é modularizado antes de qualquer extração para microsserviços. O time define fronteiras de domínio explícitas, formaliza interfaces entre os módulos e adiciona um fitness function no CI que impede um módulo de importar diretamente de outro sem passar pela interface. A modularização transforma o monólito acoplado em um monólito modular saudável, pronto para extração gradual se algum dia for necessário.

Técnicas de refatoração arquitetural

Quatro movimentos para organizar por dentro

- **Identifique os bounded contexts dentro da base acoplada**
 - Agrupe código pela linguagem do negócio, não pela técnica
- **Introduza interfaces explícitas entre os módulos**
 - Módulos conversam por contrato, nunca por acesso direto
- **Mova dados com migração cuidadosa, um contexto por vez**
 - Cada módulo passa a ser dono dos próprios dados
- **Verifique as regras de dependência automaticamente no CI**
 - A regra que não é testada volta a ser violada

Fitness function de dependência

JAVASCRIPT

```
// Falha o CI se billing importar de pedidos
import { filesOfProject } from 'tsarch';

const regra = filesOfProject()
  .inFolder('billing')
  .shouldNot()
  .dependOnFiles()
  .inFolder('pedidos');

expect(await regra).toPassAsync();
```

Extração de serviços vem depois

- Com o monólito modular, extrair serviços fica simples
- Cada módulo bem isolado já é um candidato natural

Extraia um serviço só quando houver um motivo real: escala independente, time dedicado ou ciclo de deploy próprio. Modular por dentro já entrega a maior parte do benefício, sem o custo da rede.

- Distribuir é uma decisão de negócio, não de moda



O monólito agora é modular, os serviços foram extraídos e os dados migraram. Mas tudo isso aconteceu enquanto o produto precisava continuar crescendo. E aí mora a tensão final da modernização: como você paga débito técnico sem congelar a entrega de features que o negócio espera?

Equilibrar modernização e entrega de valor